



UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIA E TECNOLOGIA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
DCC703 - COMPUTAÇÃO GRÁFICA



GEORGE LUCAS MONCAO ZAMBONIN

RELATÓRIO TRABALHO PRIMEIRO, SEGUNDO E TERCEIRO
TRABALHO

BOA VISTA, RR
2025

GEORGE LUCAS MONCAO ZAMBONIN

**RELATÓRIO TRABALHO PRIMEIRO, SEGUNDO E TERCEIRO
TRABALHO**

Relatório da disciplina de Computação Gráfica. Neste documento, são apresentadas informações sobre preenchimento, rasterização de linhas e rasterização de circunferências.

Professor: Luciano Ferreira Silva

BOA VISTA, RR

2025

1. INTRODUÇÃO

Este relatório apresenta as principais funções implementadas ao longo da disciplina de Computação Gráfica, com foco em técnicas essenciais para a geração e manipulação de gráficos em 2D. O estudo abrange algoritmos de rasterização de linhas e circunferências, além de métodos de preenchimento de áreas, fundamentais para a renderização de formas geométricas em ambientes digitais.

Os algoritmos de rasterização de linhas analisados são:

- Método Analítico;
- DDA (Digital Differential Analyzer);
- Bresenham.

Para a rasterização de circunferências, foram estudadas as seguintes abordagens:

- Equação Paramétrica;
- Simetria Incremental;
- Algoritmo de Bresenham.

Além disso, foram implementados e testados dois métodos de preenchimento de regiões:

- Flood Fill;
- Varredura Geométrica.

2. ALGORITMOS

A computação gráfica desempenha um papel fundamental na geração e manipulação de imagens digitais. Dois processos essenciais para a construção de elementos gráficos em 2D são a rasterização e o preenchimento.

A rasterização é a técnica utilizada para converter representações geométricas contínuas, como linhas e circunferências, em uma grade discreta de pixels. Esse processo é essencial para a exibição de imagens em dispositivos digitais, garantindo que os elementos gráficos sejam representados de forma clara e eficiente. Diferentes algoritmos, como os métodos analítico, DDA e Bresenham para linhas, e a equação paramétrica, o algoritmo incremental com simetria e o algoritmo de Bresenham para circunferências, são utilizados para otimizar essa conversão.

O preenchimento, por sua vez, complementa a rasterização ao determinar quais áreas internas de uma forma devem ser coloridas. Técnicas como Flood Fill e Varredura Geométrica são amplamente empregadas para garantir que regiões fechadas sejam

corretamente preenchidas, sendo fundamentais na renderização de polígonos e outras formas complexas.

Ambos os processos são indispensáveis para o desenvolvimento de gráficos computacionais, sendo amplamente aplicados em jogos, modelagem, animações e interfaces gráficas.

2.1. RASTERIZAÇÃO DE LINHAS

A rasterização de linhas é um processo essencial na computação gráfica, responsável por converter representações vetoriais de linhas em uma grade de pixels. Esse processo determina quais pixels devem ser ativados para representar a linha de forma precisa, considerando sua posição, inclinação e espessura.

Essa técnica é amplamente utilizada em monitores, impressoras e outros dispositivos gráficos, possibilitando a renderização de elementos como textos, gráficos e imagens em sistemas digitais. Para a implementação da rasterização neste trabalho, foram empregados três algoritmos distintos:

- Método Analítico;
- DDA (Digital Differential Analyzer);
- Bresenham.

Cada um desses métodos apresenta características específicas em termos de eficiência computacional e precisão, sendo analisados detalhadamente ao longo do relatório.

2.1.1 Método analítico

O método analítico realiza a rasterização de uma linha de maneira intuitiva, utilizando a equação da reta. Seu funcionamento baseia-se na determinação dos pontos pertencentes à linha por meio da equação $y=mx+b$, onde m é o coeficiente angular e b é o coeficiente linear.

O algoritmo pode ser descrito da seguinte forma:

1. Inicialmente, verifica-se se a linha é vertical ($x_1==x_2$). Caso seja, os pontos são preenchidos diretamente no intervalo de y_1 a y_2 .
2. Se a linha não for vertical, a equação da reta é determinada utilizando a inclinação m e o coeficiente b .
3. Em seguida, os pontos y são calculados para cada valor de x , e a linha é rasterizada no frame buffer.

A implementação do método analítico é apresentada a seguir:

```

void rtz::Line::analytical(int x1, int y1, int x2, int y2,
arr::Array2d frame_buffer, char value){
    char** array = frame_buffer.data;
    int rows = frame_buffer.rows;
    int cols = frame_buffer.cols;

    if (x1 == x2){
        if (y2 < y1)
            swap(&y1, &y2);

        for(int i = y1; i <= y2; i++){
            array[i][x1] = value;
        }
        return;
    }
    // find line equation
    float m = (float)(y2 - y1) / (x2 - x1);
    int b = (int)(y1 - m * x1);

    // x1 to x2 variation
    int y;
    for(int x = x1; x <= x2; x++){
        y = (int)(x * m + b);
        array[y][x] = value;
    }
    return;
}

```

```

Analytical
0 0 0 0 1
0 0 0 0 0
0 0 0 0 0
0 0 0 1 0
0 0 0 0 0
0 0 1 0 0
0 0 0 0 0
0 1 0 0 0
0 0 0 0 0
1 0 0 0 0

```

Embora seja uma abordagem simples, o método analítico apresenta algumas desvantagens significativas:

- Baixo desempenho: O uso de operações com números de ponto flutuante torna o algoritmo mais lento em comparação com outras abordagens, como o DDA e Bresenham.

- Descontinuidade da linha: Em algumas situações, devido ao arredondamento dos valores de y, a linha pode apresentar falhas ou descontinuidades, prejudicando a qualidade da renderização.

Essas limitações motivam a busca por algoritmos mais eficientes e precisos, como os que serão apresentados nas próximas seções.

2.1.2 DDA: Analisador Diferencial Digital

O Analisador Diferencial Digital (Digital Differential Analyzer – DDA) é um algoritmo de rasterização de linhas mais eficiente que o método analítico, pois reduz o número de operações de multiplicação e divisão. Além disso, ele resolve o problema de descontinuidade presente no método analítico ao considerar a variação entre as coordenadas x e y.

O algoritmo DDA verifica qual das variáveis (x ou y) possui a maior variação e, com base nessa análise, determina qual delas será usada para os incrementos principais. A variável com maior variação será incrementada em passos de 1, enquanto a outra variável será ajustada com incrementos fracionários (valores entre 0 e 1), garantindo uma distribuição mais uniforme dos pontos ao longo da linha.

A implementação do algoritmo DDA é apresentada a seguir:

```
void rtz::Line::dda(int x1, int y1, int x2, int y2, arr::Array2d
frame_buffer, char value){
    char** array = frame_buffer.data;
    int rows = frame_buffer.rows;
    int cols = frame_buffer.cols;

    int d_x, d_y;
    d_x = x2 - x1;
    d_y = y2 - y1;

    float increment;

    if (abs(d_x) > abs(d_y)){
        float y;

        increment = (float) d_y / d_x;
        y = (float)y1;

        for(int x = x1; x <= x2; x++){
            printf("aqui1");
            array[(int)y][x] = value;
            y += increment;
        }
    }
}
```

```

    }

    return;
}

increment = (float) d_x / d_y;
float x = (float) x1;

for(int y = y1; y <= y2; y++){
    printf("aqui12");
    array[y][(int)x] = value;
    x += increment;
}
}

```

```

DDA
0 0 0 0 1
0 0 0 1 0
0 0 0 1 0
0 0 1 0 0
0 0 1 0 0
0 1 0 0 0
0 1 0 0 0
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0

```

O DDA resolve o problema de descontinuidade presente no método analítico. Apresenta um desempenho superior ao reduzir o número de operações de multiplicação e divisão. Porém ainda requer o uso de números de ponto flutuante, o que pode impactar o desempenho em algumas arquiteturas. Conversão de valores de ponto flutuante para inteiros pode introduzir pequenas imprecisões.

O algoritmo DDA representa um avanço significativo em relação ao método analítico, mas seu desempenho pode ser ainda mais otimizado com o uso do algoritmo de Bresenham, que será discutido na próxima seção.

2.1.3 Algoritmo de Bresenham

O algoritmo de Bresenham resolve as principais limitações dos métodos anteriores, eliminando o uso de multiplicações complexas, números de ponto flutuante e descontinuidades na rasterização de linhas. Ele utiliza operações inteiras, tornando sua execução mais rápida e eficiente.

O algoritmo determina quais pixels devem ser ativados para representar uma linha entre dois pontos, levando em consideração sua inclinação. Ele toma decisões discretas a cada iteração para escolher os pixels mais próximos à linha ideal.

A técnica baseia-se no cálculo incremental das diferenças entre as coordenadas x e y, minimizando erros de arredondamento e otimizando a renderização. Em vez de multiplicações e divisões, Bresenham utiliza apenas somas, subtrações e multiplicações por 2, que são computacionalmente eficientes em sistemas binários.

A seguir, a implementação do algoritmo:

```
void rtz::Line::bresenham(int x1, int y1, int x2, int y2,
arr::Array2d frame_buffer, char value) {
    char** array = frame_buffer.data;
    int rows = frame_buffer.rows;
    int cols = frame_buffer.cols;
    if (x2 < x1) {
        swap(&x1, &x2);
        swap(&y1, &y2);
    }
    bool y1_bigger = false;
    if (y2 < y1) {
        y1 = -y1;
        y2 = -y2;
        y1_bigger = true;
    }
    int d_x, d_y, y, x;
    d_x = x2 - x1;
    d_y = y2 - y1;
    y = y1;
    x = x1;
    if (d_y > d_x) {
        int p = 2 * d_x - d_y;
        for (y = y1; y <= y2; y++) {
            y1_bigger ? array[-y][x] = value : array[y][x] = value;
            if (p >= 0) {
                x += 1;
                p = p + 2 * (d_x - d_y);
                continue;
            }
            p = p + 2 * d_x;
        }
        return;
    }
    int p = 2 * d_y - d_x;
```

```

for (int x = x1; x <= x2; x++) {
    y1_bigger ? array[-y][x] = value : array[y][x] = value;
    if (p >= 0) {
        y += 1;
        p = p + 2 * (d_y - d_x);
        continue;
    }
    p = p + 2 * d_y;
}
}

```

Bresenham

```

0 0 0 0 1
0 0 0 0 1
0 0 0 1 0
0 0 0 1 0
0 0 1 0 0
0 0 1 0 0
0 1 0 0 0
0 1 0 0 0
1 0 0 0 0
1 0 0 0 0

```

O algoritmo de bresenham apresenta maior eficiência, usando apenas operações inteiras (adições e subtrações), tornando a execução mais rápida. Possui rasterização precisa, evitando erros de arredondamento comuns no método analítico e no DDA e suas linhas são uniformes: O uso de cálculos diferenciais melhora a suavidade da linha rasterizada.

Em comparação com os algoritmos anteriores, o método de Bresenham é a escolha mais eficiente para a renderização de linhas em sistemas gráficos, sendo amplamente utilizado em softwares e hardware gráficos.

2.2. RASTERIZAÇÃO DE CIRCUNFERÊNCIAS

A rasterização de circunferências é o processo de converter representações geométricas contínuas de circunferências, definidas por equações matemáticas, em imagens compostas por uma grade de pixels. O objetivo é ativar os pixels corretos para representar a circunferência com precisão, considerando sua posição, raio e a resolução da imagem.

Para essa conversão, diversos algoritmos são utilizados, sendo um dos mais conhecidos o algoritmo de Bresenham. A rasterização de circunferências é essencial na renderização gráfica e tem ampla aplicação em gráficos computacionais, jogos, design gráfico e visualização de dados.

Foram analisados três algoritmos distintos: equação paramétrica, algoritmo incremental com simetria e circunferência de Bresenham.

2.2.1 Equação Paramétrica

O algoritmo da equação paramétrica permite o cálculo preciso de curvas, reduzindo falhas na rasterização. Ele utiliza funções paramétricas para definir a posição dos pontos da circunferência em termos de um parâmetro variável, como um ângulo.

A equação paramétrica da circunferência define que cada ponto da curva pode ser calculado a partir das funções seno e cosseno, utilizando o raio e a posição do centro. O ângulo varia de 0° a 360° , determinando os pontos ao longo da curva.

O algoritmo segue os seguintes passos: primeiro, percorre ângulos de 1° em 1° , convertendo-os para radianos. Em seguida, calcula as coordenadas x e y do ponto correspondente na circunferência. Esses valores são então arredondados e plotados na matriz de pixels.

A implementação do algoritmo pode ser observada no código abaixo:

```
void rtz::Circumference::parametric_equation(int x_c, int y_c, int
radius, arr::Array2d frame_buffer, char value) {
    int rows, cols;
    rows = frame_buffer.rows;
    cols = frame_buffer.cols;

    if (x_c + radius >= cols || x_c - radius < 0 || y_c + radius >=
rows || y_c - radius < 0){ // enchant it
        throw std::invalid_argument("Circumference out of bound.");
        return;
    }

    int t;
    float x, y, t_radians;
    float pi_180 = (float) PI / 180.f;

    for(t = 1; t <= 360; t++) {
        t_radians = (float) t * pi_180;
        x = x_c + radius * cosf(t_radians);
        y = y_c + radius * sinf(t_radians);
        frame_buffer.data[(int)round(y)][(int)round(x)] = value;
    }
}
```

```

Parametric
0 0 0 0 1 1 1 1 1 1 1 0 0 0 0
0 0 0 1 1 0 0 0 0 0 1 1 0 0 0
0 0 1 1 0 0 0 0 0 0 0 1 1 0 0
0 1 1 0 0 0 0 0 0 0 0 0 1 1 0
1 1 0 0 0 0 0 0 0 0 0 0 1 1
1 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 1
1 1 0 0 0 0 0 0 0 0 0 0 1 1
0 1 1 0 0 0 0 0 0 0 0 1 1 0
0 0 1 1 0 0 0 0 0 0 1 1 0 0
0 0 0 1 1 0 0 0 0 1 1 0 0 0
0 0 0 0 1 1 1 1 1 1 1 0 0 0

```

Esse método tem a vantagem de ser simples de implementar e estar diretamente baseado na equação matemática da circunferência. No entanto, apresenta algumas limitações, como o uso de funções trigonométricas, que podem ser custosas computacionalmente, além de gerar pequenos espaços entre os pontos da circunferência devido a arredondamentos. Para resoluções mais baixas, a rasterização pode apresentar descontinuidades, tornando necessário o uso de técnicas mais eficientes, como o algoritmo incremental com simetria.

2.2.2 Algoritmo Incremental com Simetria

O algoritmo incremental com simetria é baseado na divisão da circunferência em oito partes iguais, aproveitando a simetria da figura para reduzir os cálculos necessários. Em vez de calcular todos os pontos da circunferência diretamente, ele calcula apenas os pontos de um dos oitavos e espelha os resultados para os demais. Esse método reduz significativamente a complexidade computacional e melhora o desempenho da rasterização.

A ideia principal do algoritmo é utilizar deslocamentos angulares pequenos e constantes para calcular os pontos da circunferência a partir de um ponto inicial. O cálculo dos novos pontos é realizado por meio de operações de rotação incremental, utilizando seno e cosseno de um ângulo fixo.

O código abaixo implementa esse método:

```

void rtz::Circumference::simmetric_incremental(int x_center, int
y_center, int radius, arr::Array2d frame_buffer, char value, int
teta){
    int rows, cols;
    rows = frame_buffer.rows;
    cols = frame_buffer.cols;

    char** array = frame_buffer.data;

    float x = (float)radius;

```

```

float y = 0;
float teta0 = 1.f / radius;
std::cout << teta0;
float C = cosf(teta0);
float S = sinf(teta0);
int x_temp;
while (y <= x) {
    simmetric_dot(array, x_center, y_center, (int)x, (int)y);
    x_temp = x;
    x = x * C - y * S;
    y = y * C + x_temp * S;
}
}

```

Para calcular os pontos, primeiro se calcula o cosseno e seno de 1° , depois calculasse o ponto inicial, depois disso são aplicados os deslocamentos, dados por:

$$x_{k+1} = x_k \cdot \cos \theta - y_k \cdot \text{sen} \theta$$

$$y_{k+1} = y_k \cdot \cos \theta + x_k \cdot \text{sen} \theta$$

Após calcular os pontos em um arco de 45 graus, o algoritmo replica esses pontos para os demais segmentos da circunferência, utilizando a simetria. O método para aplicar essa simetria está representado no código a seguir.

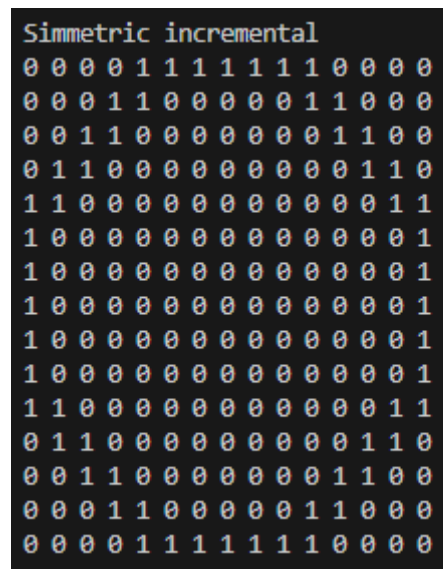
```

void rtz::Circumference::simmetric_dot(char** array, int x_center,
int y_center, int x_k, int y_k, char value){
    array[y_center + (int)y_k][x_center + (int)x_k] = value;
    array[y_center + (int)y_k][x_center - (int)x_k] = value;
    array[y_center - (int)y_k][x_center + (int)x_k] = value;
    array[y_center - (int)y_k][x_center - (int)x_k] = value;

    array[y_center + (int)x_k][x_center + (int)y_k] = value;
    array[y_center + (int)x_k][x_center - (int)y_k] = value;
    array[y_center - (int)x_k][x_center + (int)y_k] = value;
    array[y_center - (int)x_k][x_center - (int)y_k] = value;
}

```

Resultado da execução:



O uso da simetria reduz o número de cálculos necessários para rasterizar a circunferência completa, tornando o algoritmo mais eficiente do que o método da equação paramétrica. No entanto, essa abordagem ainda apresenta erros acumulativos devido ao uso de valores arredondados de x e y em cada iteração, o que pode levar a pequenas imprecisões na forma final da circunferência.

2.2.3 Circunferência de Bresenham

O algoritmo de Bresenham para rasterização de circunferências utiliza a simetria da figura para reduzir a quantidade de cálculos necessários. Diferente de outros métodos, ele evita o uso de operações como raízes, potências e funções trigonométricas, o que o torna mais eficiente e adequado para implementações em hardware gráfico e sistemas embarcados.

A ideia central do algoritmo é escolher, a cada iteração, entre três possíveis pixels vizinhos para desenhar a circunferência, selecionando aquele que está mais próximo da curva ideal. Para isso, ele calcula um fator de decisão baseado na diferença entre os pontos vizinhos e a posição teórica da circunferência. Esse fator, chamado de p , determina se o próximo pixel será deslocado apenas na direção horizontal ou também na vertical, garantindo que a circunferência seja formada da maneira mais precisa possível.

A implementação do algoritmo é a seguinte:

```
void rtz::Circumference::bresenham(int x_center, int y_center, int
radius, arr::Array2d frame_buffer, char value) {
    int rows, cols;
    rows = frame_buffer.rows;
    cols = frame_buffer.cols;
    char** array = frame_buffer.data;
```

```

int x, y;

x = 0;
y = radius;

int p = 1 - radius;

while (x <= y) {
    simmetric_dot(array, x_center, y_center, x, y, value=value);

    if (p >= 0) {
        y = y - 1;
        p = p + 2*(x - y) + 5;
        x = x + 1;
        continue;
    }

    // não altera y;
    p = p + 2*x + 3;
    x = x + 1;
}
simmetric_dot(array, x_center, y_center, x, y, value=value);
}

```

Assim como no algoritmo incremental com simetria, a função `simmetric_dot` é utilizada para refletir os pontos calculados para os outros sete segmentos da circunferência, aproveitando a simetria da figura.

A principal vantagem do algoritmo de Bresenham é que ele evita cálculos complexos e trabalha apenas com operações simples, como somas e subtrações. Isso reduz a carga computacional e melhora a eficiência da rasterização, tornando-o ideal para aplicações gráficas em tempo real. Além disso, sua abordagem incremental minimiza erros acumulativos e resulta em um traçado uniforme da circunferência.

```

Bresenham
0 0 0 0 0 1 1 1 1 1 0 0 0 0 0
0 0 0 1 1 0 0 0 0 0 1 1 0 0 0
0 0 1 0 0 0 0 0 0 0 0 0 1 0 0
0 1 0 0 0 0 0 0 0 0 0 0 0 1 0
0 1 0 0 0 0 0 0 0 0 0 0 0 1 0
1 0 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0 0 0 0 0 0 1 0
0 1 0 0 0 0 0 0 0 0 0 0 0 1 0
0 0 1 0 0 0 0 0 0 0 0 0 1 0 0
0 0 0 1 1 0 0 0 0 0 1 1 0 0 0
0 0 0 0 0 1 1 1 1 1 0 0 0 0 0

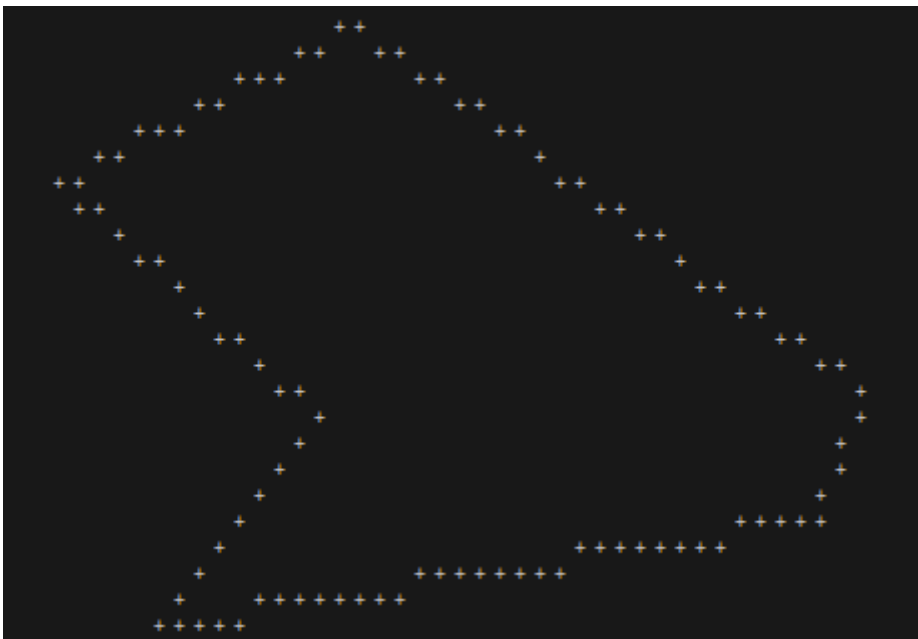
```

2.3. PREENCHIMENTO

Os algoritmos de preenchimentos são necessários em computação gráfica, pois permitem diversas aplicações, como o colorimento de formas geométricas. Estes algoritmos se baseiam nas bordas destas formas geométricas. Para aplicação da técnica, neste trabalho foram utilizados dois algoritmos distintos: Flood Fill e análise geométrica.

Ambos algoritmos utilizam as figuras a seguir para preenchimento:

```
polygon.add_vertice(5,7);  
polygon.add_vertice(20,1);  
polygon.add_vertice(45,15);  
polygon.add_vertice(43,20);  
polygon.add_vertice(10,24);  
polygon.add_vertice(18,16);
```



```
polygon.add_vertice(2, 8);  
polygon.add_vertice(25, 1);  
polygon.add_vertice(47, 5);  
polygon.add_vertice(49, 15);  
polygon.add_vertice(23, 19);  
polygon.add_vertice(20, 9);  
polygon.add_vertice(28, 7);  
polygon.add_vertice(28, 12);  
polygon.add_vertice(5, 14);
```



2.3.1 Flood Fill

O Flood Fill é um algoritmo de preenchimento utilizado para colorir áreas conexas em uma imagem ou matriz de pixels. Ele se baseia na cor inicial de um ponto semente e preenche todos os pixels vizinhos que compartilham essa mesma cor. O método é amplamente empregado em editores de imagem, jogos e outras aplicações gráficas para preencher regiões delimitadas de forma automática.

O algoritmo pode ser implementado de forma recursiva, onde ele verifica se o pixel atual tem a mesma cor da semente e, se for o caso, altera sua cor e chama recursivamente a função para os pixels vizinhos. A seguir, um exemplo de implementação:

```
void Fill::flood_fill_recursive(int x, int y, arr::Array2d
frame_buffer, char seed_color, char new_color){
    int rows, cols;
    char** array;
    rows = frame_buffer.rows;
    cols = frame_buffer.cols;
    array = frame_buffer.data;

    if (x >= cols || x < 0 || y >= rows || y < 0)
        return;

    if (array[y][x] == seed_color) {
        // array[y][x] == -1; // view effect
        array[y][x] = new_color;
    }
}
```

```

        this->flood_fill_recursive(x+1, y, frame_buffer, seed_color,
new_color);
        this->flood_fill_recursive(x-1, y, frame_buffer, seed_color,
new_color);
        this->flood_fill_recursive(x, y+1, frame_buffer, seed_color,
new_color);
        this->flood_fill_recursive(x, y-1, frame_buffer, seed_color,
new_color);
    }
}

void Fill::flood_fill(int x, int y, arr::Array2d frame_buffer, char
new_color){

    int rows, cols;
    char** array;
    rows = frame_buffer.rows;
    cols = frame_buffer.cols;
    array = frame_buffer.data;

    if (x >= cols || x < 0 || y >= rows || y < 0) {
        throw std::invalid_argument("Flood_fill coordinates out of
bound.");
        return;
    }

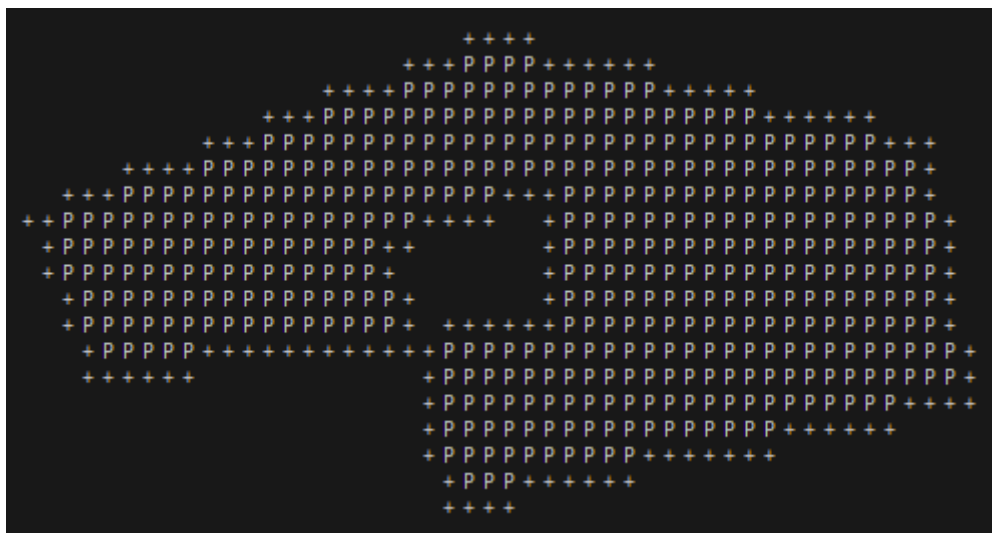
    int seed_color = array[y][x];

    this->flood_fill_recursive(x, y, frame_buffer, seed_color,
new_color);
}

```

O Flood Fill pode apresentar alguns problemas, como a limitação de preenchimento em determinadas áreas. Por exemplo, quando há espaços onde os pixels vizinhos não compartilham exatamente a mesma cor da semente, algumas partes podem permanecer sem preenchimento. Esse fenômeno pode ser observado em regiões onde pequenos artefatos gráficos ou variações de cor impedem a continuidade da recursão.

Além disso, a abordagem recursiva pode levar a estouro de pilha quando utilizada em imagens grandes, devido ao alto número de chamadas aninhadas. Para evitar esse problema, é comum utilizar uma implementação baseada em pilha ou fila para simular a recursão de maneira iterativa, otimizando o desempenho e garantindo que áreas maiores sejam preenchidas corretamente.



2.3.2 Varredura com Análise Geométrica

O algoritmo de varredura com análise geométrica é uma técnica utilizada para preenchimento de regiões em gráficos computacionais. Ele funciona identificando pares de interseções ao longo de linhas horizontais dentro de uma figura e preenchendo os pixels entre esses pares com um valor específico. Essa abordagem é particularmente útil para renderização de polígonos e outras formas geométricas fechadas.

A implementação do algoritmo percorre as linhas horizontais da imagem, identificando mudanças de cor que indicam o início e o fim de uma região a ser preenchida. Quando um par de interseções é encontrado, os pixels entre esses dois pontos são modificados para a cor desejada. Abaixo está o código do algoritmo:

```
void Fill::geometric(int x_min, int y_min, int x_max, int y_max,
arr::Array2d frame_buffer, char color) {
```

```

// Acesso aos dados do frame buffer
char** array = frame_buffer.data;
int rows = frame_buffer.rows;
int cols = frame_buffer.cols;

// Variáveis auxiliares
int x, y, x_min_pair;
bool pair;

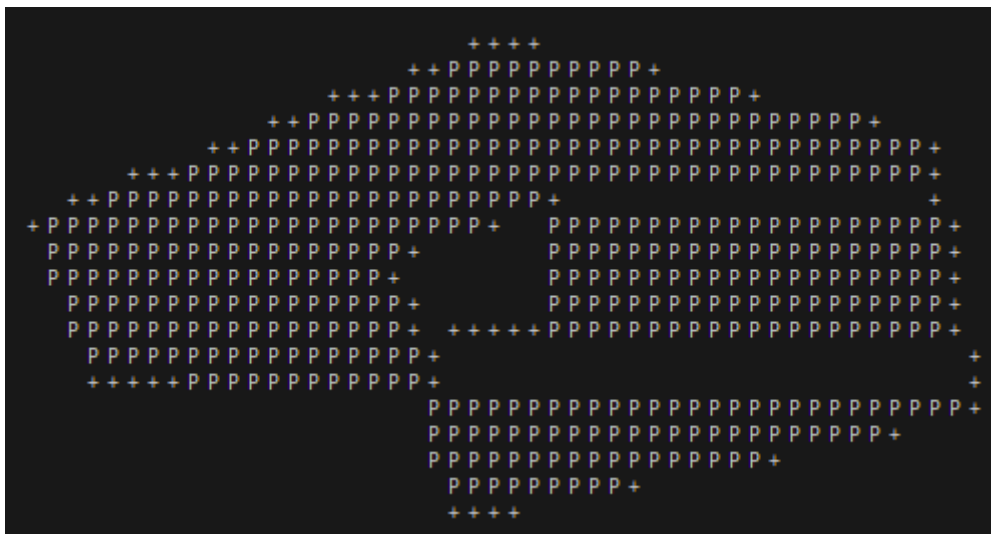
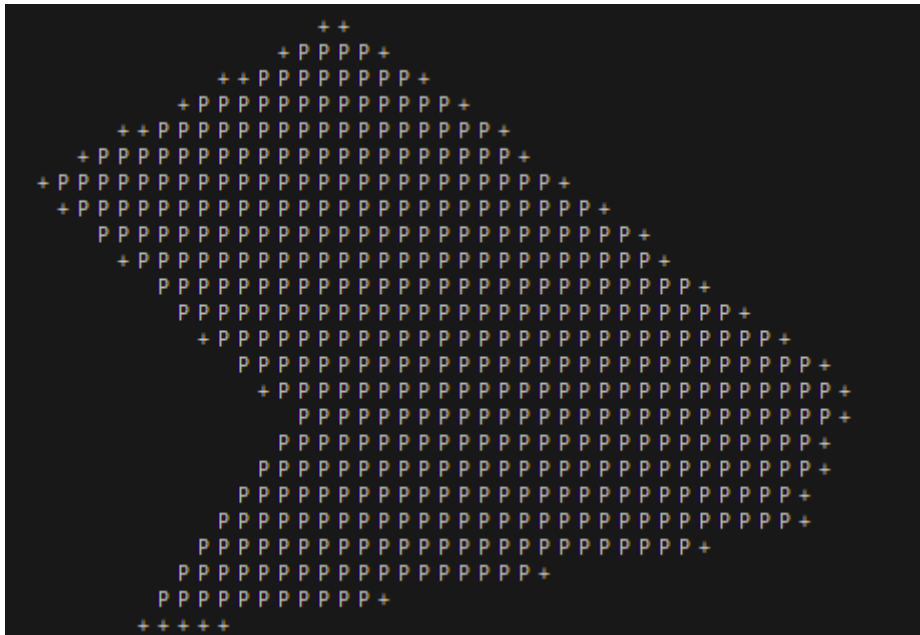
// Percorre as linhas do y_min até y_max
for (y = y_min; y <= y_max; y++) {
    x = x_min;
    pair = false;

    // Percorre as colunas da linha atual
    while (x <= x_max) {
        if (array[y][x] == color) {
            if (pair) {
                while(x <= x_max){
                    if (array[y][x+1] == color){
                        x++;
                    } else {
                        break;
                    }
                }
                pair = false;
                for (; x_min_pair < x; x_min_pair++) {
                    array[y][x_min_pair] = color;
                }
            } else {
                while(x <= x_max){
                    if (array[y][x+1] == color){
                        x++;
                    } else {
                        break;
                    }
                }
                pair = true;
                x_min_pair = x;
                x++;
            }
        }
        x++;
    }
}

```

}

Segue os resultados da execução do algoritmo nas figuras dadas anteriormente:



A execução do algoritmo pode gerar algumas falhas no preenchimento, principalmente em figuras com contornos complexos ou bordas irregulares. Como a detecção das interseções depende da varredura de linhas horizontais, algumas regiões podem ser preenchidas de maneira incorreta devido a interpretações equivocadas das interseções. Isso pode resultar tanto em preenchimentos indevidos quanto em lacunas dentro da figura.

Para melhorar a precisão do algoritmo, podem ser aplicadas técnicas de refinamento, como a análise mais detalhada dos contornos ou o uso de algoritmos híbridos que combinem a varredura geométrica com outros métodos de preenchimento, garantindo um resultado mais consistente.

3. CONCLUSÃO

Em suma, os algoritmos de rasterização são essenciais na computação gráfica, pois transformam representações matemáticas em imagens digitais com precisão e eficiência. Dentre eles, o algoritmo de Bresenham se destaca pela otimização no desenho de linhas e circunferências, reduzindo erros de arredondamento e eliminando operações complexas. Além disso, técnicas como o DDA, Flood Fill e varredura com análise geométrica oferecem abordagens diversas para a renderização e preenchimento de formas, cada uma com suas vantagens e limitações.

Esses algoritmos são amplamente utilizados em aplicações que vão desde jogos eletrônicos e design gráfico até sistemas de visualização de dados, garantindo uma representação fiel e eficiente das imagens. O domínio dessas técnicas é fundamental para profissionais da área, permitindo a criação de gráficos mais otimizados e visualmente refinados, contribuindo para o avanço contínuo da computação gráfica e suas aplicações.